Javascript

목 차

- JavaScript 기본 문법
- DOM과 이벤트
- Web API

과정 목표

: JavaScript 기초 문법부터 DOM·이벤트까지 이해하고, 간단한 웹 기능을 스스로 구현할 수 있는 기초 역량을 기른다.

JavaScript란?

웹 브라우저에서 동작하는 프로그래밍 언어

- 버튼 클릭, 입력 처리, 화면 변경 담당
- **HTML:** 구조 / **CSS:** 디자인 / **JS:** 동작

JavaScript 실행 방법

1. 브라우저 개발자 도구(Console)
   브라우저에 즉시 JS를 실행해보는 공간 → 바로 실행되고 결과 확인 가능

용도

- 문법 연습
- 값 확인
- 에러 확인

```
console.log(1 + 2);
```

JavaScript 실행 방법

2. **<script> 태그**
   HTML 안에서 JavaScript를 실행하겠다고 선언하는 태그
   브라우저가 HTML을 읽다가 <script>를 만나면 JS 실행

```
<script>
  console.log("Hello");
</script>
```

JavaScript 실행 방법

3. **HTML 파일과 JS 파일 분리**
   JavaScript 코드를 별도 파일(.js)로 관리

```
<script src="app.js"></script>


// app.js
console.log("분리된 JS 파일");
```

JavaScript 실행 방법

4. **Node.js 실행**
   터미널에서 **JavaScript** 파일 실행
   브라우저 없이 **JS** 실행 가능

```
$ node app.js
```

Node.js 설치

- macOS: brew install node
- Windows: https://nodejs.org/ko/download/current

```
$ node -v
$ npm -v
```

- npm: Node.js용 패키지 관리자

Node.js 장점

1. 환경이 단순
   - HTML, CSS 없이 JavaScript만 실행
2. 실행이 간단
   - node app.js 한 줄로 실행
3. 에러 메시지가 명확
   - 스택 트레이스가 직관적이라 원인 추적이 쉬움
4. 초보자에게 친절
   - 문법 오류를 바로 확인 가능

변수란?

값을 저장하는 공간

- 한 번 저장한 값을 재사용 가능

```
let age = 20;
const username = "Alex";


console.log(age);
console.log(username);
```

let vs. const

- **let: 값 변경 가능**
- **const: 값 변경 불가(기본 권장)**

```
let count = 1;
count = 2;


const pi = 3.14;
// pi = 3.15 ❌
```

자료형 (Data Type)

- number
- string
- boolean
- null
- undefined

number

숫자를 나타내는 자료형
정수, 실수 구분 없이 모두 **number**

```
let a = 10;
let b = 3.14;


console.log(a + b);
console.log(typeof a);
```

string

문자열 자료형
따옴표(" " 또는 ' ')로 감싼 값

```
let username = "Alex";
let message = 'Hello';


console.log(username);
console.log(message);
console.log(typeof username);
```

boolean

참(true) / 거짓(false)을 나타내는 자료형
조건문에서 주로 사용

```
let isStudent = true;
let hasTicket = false;


console.log(isStudent);
console.log(typeof isAdult);
```

null

값이 의도적으로 비어 있음을 의미
개발자가 직접 설정

```
let selectedUser = null;


console.log(selectedUser);
console.log(typeof selectedUser); // object (JS의 오래된 특징)
```

undefined

값이 아직 할당되지 않음을 의미
자동으로 설정됨

```
let score;


console.log(score);
console.log(typeof score); // undefined
```

null vs. undefined

- **null:** 비어 있다고 명시
- **undefined:** 아직 정해지지 않음

```
let a = null;
let b;

console.log(a == b);   // true
console.log(a === b);  // false
```

연산자(Operator)

값을 계산하거나 비교할 때 사용하는 기호

- 산술 연산자
- 비교 연산자
- 동등 / 일치 연산자
- 논리 연산자
- 문자열 연산자

산술 연산자

숫자 계산에 사용

```
let a = 10;
let b = 3;

console.log(a + b); // 더하기
console.log(a - b); // 빼기
console.log(a * b); // 곱하기
console.log(a / b); // 나누기
```

비교 연산자

값의 크기나 같음을 비교
결과는 **boolean**

```
console.log(10 > 5);
console.log(10 < 5);
console.log(10 >= 10);
console.log(10 <= 9);
```

동등 / 일치 연산자

- == : 값만 비교 (타입 변환 발생)
- === : 값 + 타입 비교 (권장)

```
console.log(10 == "10");   // true
console.log(10 === "10");  // false
```

논리 연산자

여러 조건을 조합

```
let age = 20;
let hasId = true;

console.log(age >= 19 && hasId); // AND
console.log(age < 19 || hasId);  // OR
console.log(!hasId);             // NOT
```

문자열 연산자

+로 문자열 연결

```
let firstName = "Alex";
let lastName = "Kim";


console.log(firstName + " " + lastName);
```

JavaScript vs. Python

JS는 타입 변환이 자동으로 일어나기 때문에, Python보다 비교 연산에서 더 주의가 필요

문자열 + 숫자

- **JavaScript:** 자동 문자열 변환 발생
- **Python:** 에러 발생

```
"10" + 1   // "101"
```

조건문(if / else)

조건이 true일 때만 코드를 실행하기 위한 문법

```
if (조건식) {
  // 조건이 true일 때 실행
} else {
  // 조건이 false일 때 실행
}
```

조건문(if / else)

```
let age = 20;


if (age >= 19) {
  console.log("성인");
} else {
  console.log("미성년자");
};
```

조건문(if / else) 특징

- 조건식의 결과는 **boolean**으로 평가
- 값 자체도 조건식으로 사용할 수 있다 **(truthy / falsy)**
  - **falsy 값: false, 0, "", null, undefined, NaN (6개 고정)**
  - **truthy 값: falsy를 제외한 모든 값**
    - 빈 배열 **[]**, 빈 객체 **{}**도 **truthy**
    - 문자열 **"0"** 은 숫자 **0**이 아니라 문자열이라 **truthy**
- **{}** 블록 단위로 실행 범위 결정
- **else**는 선택 사항

실습 1: 콘솔 계산기

프로그램 동작

1. 숫자 두 개를 변수로 선언한다
2. 연산자를 문자열로 정한다 ("+", "-", "*", "/")
3. 연산자에 따라 계산 결과를 출력한다

⚠️ 사용자로부터 입력은 직접 받지 않고, 변수로 선언한다.

실습 1: 콘솔 계산기

```
let a = 10;
let b = 5;
let operator = "+"; // "+", "-", "*", "/"
```

```
if (operator === "+") {
  console.log(a + b);
} else if (operator === "-") {
  console.log(a - b);
} else if (operator === "*") {
  console.log(a * b);
} else if (operator === "/") {
  console.log(a / b);
} else {
  console.log("지원하지 않는 연산자입니다");
}
```

실습 2: 최종 합격 판정 프로그램

합격 조건

- 점수(score) 70 이상
- 출석률(attendance) 80 이상
- 과제 제출 여부(submitted) true
- 단, 점수 90 이상이면 출석률은 무시하고 합격

실습 2: 최종 합격 판정 프로그램

```
let score = 88;
let attendance = 75;
let submitted = true;

if (submitted && ((score >= 70 && attendance >= 80) || score >= 90)) {
  console.log("최종 합격");
} else {
  console.log("불합격");
}
```

퀴즈) 다음 중 Node.js의 역할로 가장 적절한 것은?

1. HTML을 꾸민다
2. JavaScript를 브라우저에서만 실행한다
3. 브라우저 없이 JavaScript를 실행한다
4. CSS를 컴파일한다

퀴즈) 다음 코드의 출력 결과는?

```
console.log(10 + "5");
```

1. 15
2. "15"
3. "105"
4. 에러 발생

퀴즈) 다음 중 **falsy** 값이 아닌 것은?

1. **0**
2. **""**
3. **[]**
4. **null**

퀴즈) == 와 === 의 차이로 옳은 설명은?

1. 차이가 없다
2. == 는 타입까지 비교한다
3. === 는 값만 비교한다
4. === 는 값과 타입을 모두 비교한다

반복문

반복 작업을 한 번에 처리

```
console.log(1);
console.log(2);
console.log(3);
```

```
for (let i = 1; i <= 3; i++) {
  console.log(i);
}
```

for 반복문

기본 구조

```
for (초기값; 조건식; 증가식) {
  // 반복 실행
}
```

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

while 반복문

조건이 **true**인 동안 반복

```
let i = 0;

while (i < 3) {
  console.log(i);
  i++;
}
```

반복문 핵심 포인트

- 조건이 **false**가 되면 종료
- 무한 루프 주의
- 반복 횟수가 정해져 있으면 **for** 권장

배열(Array)

- 여러 값을 하나로 묶은 자료형
- 순서(index)가 있음
- 요소로 서로 다른 타입을 가질 수 있음

```
let numbers = [10, 20, 30];
console.log(numbers[0]);
console.log(numbers.at(-1));
console.log(numbers[3]);
```

배열 순회

for...of

```
let scores = [80, 90, 70];


for (const score of scores) {
  console.log(score);
}
```

배열 순회

entries

- 인덱스 + 값 둘 다 필요한 경우

```
for (const [i, score] of scores.entries()) {
  console.log(i, score);
}
```

배열 순회

for문과 index로 순회

```
let scores = [80, 90, 70];


for (let i = 0; i < scores.length; i++) {
  console.log(scores[i]);
}
```

배열 순회

forEach

- 중간에 **break / continue** 불가

```
scores.forEach(score => {
  console.log(score);
});
```

객체(Object)

- 여러 값을 이름(key)으로 묶음

```
let user = {
  name: "Alex",
  age: 20,
};

console.log(user.name);
console.log(user["age"]);
```

객체(Object) 순회

```
for (const key in user) {
    console.log(key, user[key]);
};
```

배열 vs. 객체

- 배열: 순서가 중요한 데이터
- 객체: 의미 있는 이름이 중요한 데이터

맵(Map)

- **key–value** 형태의 데이터 구조
- 객체({})보다 용도가 명확

```
const map = new Map();


map.set("name", "Alex");
map.set("age", 20);


console.log(map.get("name"));
console.log(map.size);
```

객체 vs. 맵(Map)

- 객체: 문자열 key 중심
- Map: 모든 타입 key 가능, 반복 처리에 유리

```
const map = new Map([["a", 1], ["b", 2]]);

for (const [key, value] of map) {
  console.log(key, value);
}
```

함수(Function)

- 코드를 묶어서 재사용
- 입력 → 처리 → 출력

```
function add(a, b) {
  return a + b;
}

console.log(add(3, 5));
```

함수 핵심 포인트

- 함수는 호출해야 실행됨
- **return**은 결과 반환 + 함수 종료
- 매개변수는 함수 내부에서만 사용

함수를 값처럼 다루기

1. 함수를 변수에 할당할 수 있다

```
function sayHello() {
  console.log('hello');
}

const f = sayHello;

f(); // hello
```

함수를 값처럼 다루기

2. 함수를 함수의 인자로 전달할 수 있다

```
function run(fn) {
  fn();
}
run(sayHello); // hello
```

함수를 값처럼 다루기

(기본) 함수를 선언한 곳에서 직접 호출
(응용) 함수를 선언한 곳과 호출하는 곳이 달라질 수 있다

함수 전달과 실행의 차이

함수를 전달하는 것과 함수 실행 결과를 전달하는 것은 다르다

```
// 함수 전달
run(sayHi);   // hi


// 함수 실행 결과 전달
run(sayHi()); // hi 출력 후 에러 (fn이 undefined)
```

실습 1: 반복문 기초

과제: 1부터 10까지 콘솔에 출력하세요.

실습 1: 반복문 기초

과제: 1부터 10까지 콘솔에 출력하세요.

```
for (let i = 1; i <= 10; i++) {
  console.log(i);
}
```

실습 1: 반복문 기초

과제: 짝수만 출력하세요 (1~20)

실습 1: 반복문 기초

과제: 짝수만 출력하세요 (1~20)

```
for (let i = 1; i <= 20; i++) {
  if (i % 2 === 0) {
    console.log(i);
  }
}
```

실습 2: 배열 + 반복문

과제: 배열의 모든 값을 출력하세요.

```
let fruits = ["apple", "banana", "orange"];
```

실습 2: 배열 + 반복문

과제: 배열의 모든 값을 출력하세요.

```
let fruits = ["apple", "banana", "orange"];

for (const fruit of fruits) {
  console.log(fruit);
}
```

실습 2: 배열 + 반복문

과제: 배열의 모든 값을 출력하세요.

```
let fruits = ["apple", "banana", "orange"];

for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

실습 2: 배열 + 반복문

과제: 배열의 합계를 구하세요.

```
let numbers = [10, 20, 30, 40];
let sum = 0;
```

실습 2: 배열 + 반복문

과제: 배열의 합계를 구하세요.

```
let numbers = [10, 20, 30, 40];
let sum = 0;

for (const num of numbers) {
  sum += num;
}
console.log(sum);
```

실습 3: 객체 다루기

과제: 사용자 정보(이름, 나이, 이메일)를 객체로 만들고 출력하세요.

실습 3: 객체 다루기

과제: 사용자 정보(이름, 나이, 이메일)를 객체로 만들고 출력하세요.

```
let user = {
  name: "Alex",
  age: 25,
  email: "alex@test.com",
};
console.log(user.name);
console.log(user.age);
console.log(user.email);
```

실습 4: 함수 기초

과제: 두 숫자를 받아 더하는 함수를 작성하세요.

실습 4: 함수 기초

과제: 두 숫자를 받아 더하는 함수를 작성하세요.

```
function add(a, b) {
  return a + b;
}

console.log(add(3, 7));
```

실습 4: 함수 기초

과제: 점수를 받아 합격 여부를 반환하는 함수를 작성하세요.

- 합격: 60점 이상 / 불합격: 그 외

```
function checkPass(score) {
  ...
}


console.log(checkPass(75));
console.log(checkPass(50));
```

실습 4: 함수 기초

과제: 점수를 받아 합격 여부를 반환하는 함수를 작성하세요.

- 합격: 60점 이상 / 불합격: 그 외

```
function checkPass(score) {
  if (score >= 60) {
    return "합격";
  } else {
    return "불합격";
  }
}
```

실습 5: 종합 실습

점수 배열을 받아서 평균을 반환하는 함수를 작성하세요.

```
function getAverage(scores) {
  ...
}

console.log(getAverage([80, 85, 92, 97]));
```

실습 5: 종합 실습

```
function getAverage(scores) {
    if (scores.length === 0) {
        return 0;
     }
    let sum = 0;
    for (const score of scores) {
        sum += score;
    }
    return sum / scores.length;
}
```

DOM이란?

Document Object Model

- **HTML 문서를 JavaScript가 다룰 수 있게 만든 객체 구조**
- **웹 페이지 = 트리 구조**

JavaScript는 어떻게 화면을 바꾸나?

- HTML을 직접 수정하는 것이 아님
- DOM 객체를 조작해서 화면 변경

DOM 선택하기

주요 메서드

- **document.getElementById**
- **document.querySelector**
- **document.querySelectorAll**

```
<h1 id="text">Hello</h1>
```

```
const el = document.getElementById("text");
console.log(el.textContent);
console.log(el.textContent.trim());
```

스타일 변경

```
el.textContent = "Hello, DOM";
el.style.color = "red";
el.style.fontSize = "20px";
```

DOM 선택하기

querySelector

- CSS 선택자로 요소 1개 선택
- 첫 번째로 매칭된 요소만 반환(없으면 null)

```
document.querySelector("#text");
document.querySelector(".item");
document.querySelector("p");
```

DOM 선택하기

querySelectorAll

- **CSS 선택자로 요소 여러 개 선택**
- **NodeList 반환(없으면 빈 NodeList)**

```
const items = document.querySelectorAll(".item");


items.forEach(el => {
  console.log(el.textContent);
});
```

DOM 내용 변경

```
el.textContent = "Hello JavaScript";


// textContent: 텍스트만 변경
// innerHTML: HTML 포함 변경
```

속성 변경

```
const link = document.querySelector("a");
link.href = "https://google.com";
```

이벤트(Event)란?

웹 페이지에서 발생하는 모든 동작이나 상태 변화

- 주로 사용자의 행동에 의해 발생하지만, 브라우저 자체 동작도 포함

사용자 행동

- **클릭(click), 입력(input), 키보드 입력(keydown, keyup), 스크롤 (scroll)**

브라우저 동작

- **페이지 로드(load), DOM 로드 완료(DOMContentLoaded), 창 크기 변경 (resize)**

이벤트 처리 방식

JavaScript에서는 이벤트 리스너(event listener)를 등록하여 이벤트를 처리

- 특정 요소에 특정 이벤트가 발생하면 지정한 함수가 실행

```
const btn = document.querySelector("button");


btn.addEventListener("click", function () {
  console.log("클릭됨");
});
```

이벤트 리스너를 쓰는 이유

- 사용자의 행동을 감지해서 상호작용(interaction)을 구현
- 정적인 페이지를 동적인 웹 애플리케이션으로 변화
- 화면(UI)과 로직을 동적으로 제어

입력값 처리

```
<input id="name-input" />
<button>확인</button>
```

```
const input = document.querySelector("#name-input");
const button = document.querySelector("button");
button.addEventListener("click", function () {
  const p = document.createElement("p");
  p.textContent = input.value;
  document.body.appendChild(p);
});
```

입력값 처리

```
<div id="result"></div>
```

```
function printName() {
        result.innerHTML = "";
        const welcomeText = "환영합니다, " + input.value + " 님";
        const p = document.createElement("p");  // <p></p>
        p.textContent = welcomeText;  // <p>welcomeText</p>
        result.appendChild(p);
    };
```

DOM + 조건문

사용자 입력에 따라 화면과 동작을 다르게 처리

```
<button>확인</button>
```

```
function printName() {
        if (input.value.trim() === "") {
            alert("이름을 입력하세요");
            return;
        };
```

실습 1: 버튼 클릭 이벤트

과제: 버튼을 클릭하면 콘솔에 “버튼 클릭됨”을 출력하세요

```
<button id="btn">클릭</button>
```

실습 1: 버튼 클릭 이벤트

과제: 버튼을 클릭하면 콘솔에 “버튼 클릭됨”을 출력하세요

```
<button id="btn">클릭</button>
```

```
const btn = document.querySelector("#btn");

btn.addEventListener("click", function () {
  console.log("버튼 클릭됨");
});
```

실습 2: 텍스트 변경하기

과제: 버튼을 클릭하면 <p> 태그의 텍스트를 "Hello JavaScript"로 변경

```
<h1 class="text">Hello</h1>
<button id="change">변경</button>
```

실습 2: 텍스트 변경하기

과제: 버튼을 클릭하면 텍스트를 **"Hello JavaScript"**로, 색상은 빨간색으로 변경

```
const h1 = document.querySelector(".text");
const button = document.querySelector("#change");


button.addEventListener("click", function () {
  h1.textContent = "Hello JavaScript";
  h1.style.color = "red";
});
```

실습 3: 실시간 입력 감지

```
<input type="password" placeholder="비밀번호 입력">
<p id="result"></p>
```

실습 3: 실시간 입력 감지

```
function validatePassword() {
        if (input.value.length < 4) {
            result.textContent = "4자리 이상 입력하세요."
            result.style.color = "red";
        } else {
            result.textContent = "사용 가능한 비밀번호입니다."
            result.style.color = "blue";
        }
    };
```

실습 4: 마우스 감지

과제: 마우스를 올리면 텍스트가 오렌지색, 내리면 검정색이 되게 만드세요.

```
<p id="box">마우스를 올려보세요</p>
```

실습 4: 마우스 감지

과제: 마우스를 올리면 텍스트가 오렌지색, 내리면 검정색이 되게 만드세요.

```
function highlightText() {
    box.style.color = "orange";
    box.style.border = "1px solid orange";
};
```

화살표 함수

짧고 간결하게 함수를 정의하기 위한 문법

- **function** 키워드 대신 **=>** 사용
- 콜백 함수에 주로 사용

```
function add(a, b) {
  return a + b;
}
const add = (a, b) => {
  return a + b;
};
const add = (a, b) => a + b;
```

화살표 함수

- 이벤트 리스너에 사용

```
button.addEventListener("click", function () {
  console.log("클릭됨");
});

button.addEventListener("click", () => {
  console.log("클릭됨");
});
button.addEventListener("click", () => console.log("클릭됨"));
```

실습 4: 마우스 감지

과제: 마우스를 올리면 텍스트가 오렌지색, 내리면 검정색이 되게 만드세요.

```
box.addEventListener("mouseover", () => changeBackgroundColor("skyblue"));
box.addEventListener("mouseout", () => changeBackgroundColor("orange"));

function changeBackgroundColor(color) {
    box.style.backgroundColor = color;
};
```

호이스팅 (Hoisting)

코드 실행 전에 한 번 전체를 스캔하여 선언(변수·함수)을 먼저 처리하는 동작 방식

- **changeBackgroundColor**가 밑에 있어도 실행 가능한 이유

실습 5: 스타일 변경

과제: 버튼을 클릭하면, 텍스트 색상은 오렌지 & 크기는 **20px**이 되게 변경하세요.

```
<p id="box">색상 변경</p>

<button id="color">변경</button>
```

실습 5: 스타일 변경

과제: 버튼을 클릭하면, 텍스트 색상은 오렌지 & 크기는 **20px**이 되게 변경하세요.

```
const box = document.querySelector("#box");
const btn = document.querySelector("#color");


btn.addEventListener("click", function () {
  box.style.color = "orange";
  box.style.fontSize = "20px";
});
```

실습 6: 종합 실습

과제: 점수를 입력하고, 버튼을 클릭하면, 합격 / 불합격 여부를 출력하세요

- 합격: 60점 이상 / 불합격: 그외

```
<input id="score" placeholder="점수 입력" />
<button id="check">확인</button>
<p id="result"></p>
```

실습 6: 종합 실습

```
const scoreInput = document.querySelector("#score");
const button = document.querySelector("#check");
const result = document.querySelector("#result");
button.addEventListener("click", function () {
  const score = Number(scoreInput.value);
  if (score >= 60) {
    result.textContent = "합격";
  } else {
    result.textContent = "불합격";
  }
});
```

퀴즈) DOM이란?

1. HTML 파일 그 자체
2. 브라우저가 HTML을 객체로 표현한 구조
3. JavaScript 문법
4. CSS 규칙 집합

퀴즈) 다음 중 이벤트가 아닌 것은?

1. **click**
2. **input**
3. **scroll**
4. **for**

퀴즈) 다음 코드에서 콘솔에 출력되는 것은?

```
button.addEventListener("click", () => {
  console.log(input.value);
});
```

1. 버튼 텍스트
2. input 요소 객체
3. 클릭 시점의 input 값
4. undefined

Web API

브라우저가 **JavaScript**에 제공하는 기능 모음

- **JavaScript** 언어 자체가 아니라 브라우저 환경의 **API**
- 웹 페이지가 브라우저 기능과 상호작용하도록 도와줌
- 사용자 이벤트, 화면 제어, 네트워크 통신 등을 가능하게 함

Web API 예시

1. DOM API
    - document.querySelector, element.textContent
2. Event API
    - addEventListener
3. Timer API
    - setTimeout, setInterval
4. Storage API
    - localStorage, sessionStorage
5. Fetch API
    - 서버와 통신 (fetch)

Timer API

setTimeout

- 일정 시간이 지난 뒤에 함수를 한 번 실행하는 **Web API**

```
setTimeout(function () {
  console.log("3초 후 실행");
}, 3000);
```

Timer API

setInterval

- 일정 시간마다 함수를 반복 실행하는 **Web API**

```
setInterval(function () {
  console.log("1초마다 실행");
}, 1000);
```

Timer API

timerId

- **setTimeout / setInterval**이 타이머를 구분하기 위해 돌려주는 번호
- 타이머를 멈추거나 버튼 여러 번 눌러도 타이머 하나만 실행되게 하기 위해 사용

```
const timerId = setInterval(function () {
  console.log("실행");
}, 1000);


clearInterval(timerId);
```

실습: Timer API

과제: 5초 카운트 다운 후, 끝을 출력하는 타이머 만들기

```
<button id="start">카운트다운 시작</button>
<p id="result"></p>
```

```
const startBtn = document.querySelector("#start");
const timeEl = document.querySelector("#result");


let time = 5;
let timerId = null;
```

실습: Timer API

```
startBtn.addEventListener("click", function () {
    if (timerId !== null) return;
    timeEl.textContent = time;
    …
});
```

실습: Timer API

```
timerId = setInterval(function () {
    time--;
    if (time === 0) {
      timeEl.textContent = "끝";
      clearInterval(timerId);
      timerId = null;
    } else {
      timeEl.textContent = time;
    }
  }, 1000);
```

Storage API

브라우저가 제공하는 클라이언트 측 저장소(Web API)

- 서버 없이도 데이터를 브라우저에 저장/조회

특징

- 키(key) / 값(value) 형태로 데이터 저장
- 값은 문자열(string)로 저장
- 페이지 새로고침 후에도 유지

localStorage

- 영구 저장(브라우저를 껐다 켜도 남아 있음)
- 로그인 정보, 설정값 저장에 사용
- 만료 시간 없음

```
localStorage.setItem("name", "Alex");
localStorage.getItem("name");
localStorage.removeItem("name");
localStorage.clear();
```

객체 저장하기

Storage는 문자열만 저장 가능 → JSON 사용

```
const user = { name: "Alex", age: 20 };

localStorage.setItem("user", JSON.stringify(user));

const savedUser = JSON.parse(localStorage.getItem("user"));
console.log(savedUser.name);
```

sessionStorage

- 세션 단위 저장(브라우저(탭) 닫으면 삭제)
- 임시 데이터
- 같은 사이트라도 탭마다 별도

```
sessionStorage.setItem("name", "Alex");
sessionStorage.getItem("name");
sessionStorage.removeItem("name");
sessionStorage.clear();
```

실습: 입력값 저장

과제: 저장한 값이 새로고침해도 유지되도록 만드세요

```
<input id="name" placeholder="이름 입력" />
<button id="save">저장</button>
<p id="result"></p>
```

```
const input = document.querySelector("#name");
const button = document.querySelector("#save");
const result = document.querySelector("#result");
```

실습: 입력값 저장

```
const savedName = localStorage.getItem("name");
if (savedName) {
  result.textContent = savedName;
}

button.addEventListener("click", function () {
  localStorage.setItem("name", input.value);
  result.textContent = input.value;
});
```

Fetch API

브라우저가 제공하는 서버 통신용 **Web API**

- **HTTP 요청(Request)을 보내고 응답(Response)을 수신**
- 비동기로 동작

비동기(**Asyncronous**)

- 오래 걸리는 작업을 기다리지 않고, 다음 코드를 먼저 실행하는 방식(= 일시정지)
- 서버 요청은 시간이 걸리기 때문에 요청은 보내고, 다른 작업을 계속 진행
- **fetch**는 서버에 요청을 보내고, 응답이 오면 그때 결과를 처리

기본 사용법(GET)

```
fetch("https://jsonplaceholder.typicode.com/posts")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

1. **fetch(url)** → 요청 보냄
2. **response** → 서버 응답
3. **response.json()** → **JSON** 파싱
4. **data** → 실제 데이터

실습: DOM과 함께 사용하기

과제: 버튼을 누르면 posts 5개 title 조회하기

```
<button id="load">불러오기 </button>
<ul id="list"></ul>
```

```
const button = document.querySelector("#load");
const list = document.querySelector("#list");
```

실습: DOM과 함께 사용하기

과제: 버튼을 누르면 posts 5개 title 조회하기

```
button.addEventListener("click", function () {
  fetch("https://jsonplaceholder.typicode.com/posts")
    .then(res => res.json())
    .then(posts => {
      …
  });
});
```

실습: DOM과 함께 사용하기

과제: 버튼을 누르면 posts 5개 title 조회하기

```
list.innerHTML = "";

for (const post of posts.slice(0, 5)) {
  const li = document.createElement("li");
  li.textContent = post.title;
  list.appendChild(li);
});
```

POST 요청

```
<input type="email" id="email">
<button id="send">전송</button>
<pre id="result"></pre>
```

```
const input = document.querySelector("#email");
const button = document.querySelector("#send");
const result = document.querySelector("#result");


button.addEventListener("click", function () {
    …
});
```

POST 요청

```
fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({email: input.value}),
  })
    .then(res => res.json())
    .then(data => {
      result.textContent = JSON.stringify(data, null, 2);  // 2 → 들여쓰기 (가독성)
    });
```

최종 실습: To-do 앱 만들기

브라우저가 제공하는 서버 통신용 **Web API**

- **HTTP 요청(Request)을 보내고 응답(Response)을 수신**
- 비동기로 동작

최종 실습: To-do 앱 만들기

비동기(Asyncronous)

- 오래 걸리는 작업을 기다리지 않고, 다음 코드를 먼저 실행하는 방식(= 일시정지)
- 서버 요청은 시간이 걸리기 때문에 요청은 보내고, 다른 작업을 계속 진행
- **fetch**는 서버에 요청을 보내고, 응답이 오면 그때 결과 처리